

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO HỌC PHẦN
KỸ THUẬT VÀ CÔNG NGHỆ DỮ LIỆU LỚN

ĐỀ TÀI

Cảnh báo sớm nhiễm khuẩn huyết
trong ICU bằng Big Data và MLOps

Giảng viên hướng dẫn: TS. Trần Hồng Việt
ThS. Ngô Minh Hương

Nhóm sinh viên thực hiện: Lê Thiện Đức 23020358
Nguyễn Đức Huy 23020376

Hà Nội, tháng 6 năm 2026

Ý KIẾN ĐÁNH GIÁ

Ý kiến đánh giá:

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm số:

Điểm chữ:

Hà Nội, ngày ..., tháng ..., năm 20...

Giảng viên đánh giá
(Ký, ghi rõ họ tên)

Mục lục

1	Giới thiệu	4
1.1	Bối cảnh	4
1.2	Mục tiêu	4
1.3	Tổng quan giải pháp	4
2	Dữ liệu và bài toán Sepsis	5
2.1	Nguồn dữ liệu	5
2.2	Cấu trúc và đặc điểm dữ liệu	6
2.3	Tổ chức dữ liệu trong dự án	6
2.4	Bài toán dự đoán Sepsis	7
3	Phân tích và thiết kế hệ thống	7
3.1	Kiến trúc tổng thể	7
3.2	Pipeline xử lý dữ liệu thời gian thực	8
3.3	Pipeline phục vụ dữ liệu và giám sát	9
3.4	Công nghệ sử dụng	10
3.4.1	Apache NiFi	10
3.4.2	Apache Kafka	10
3.4.3	Apache Spark Streaming	11
3.4.4	Apache Cassandra	11
3.4.5	HDFS	12
3.4.6	Flask API	12
3.4.7	Apache Airflow	13
3.4.8	MLflow	13
3.4.9	Docker Compose	14
4	Quy trình xử lý và phân tích dữ liệu	15
4.1	Tổng quan luồng xử lý dữ liệu	15
4.2	Thu thập và chuyển đổi dữ liệu đầu vào	15
4.3	Tiền xử lý dữ liệu trong Spark Streaming	16
4.4	Tạo đặc trưng theo chuỗi thời gian	16
4.5	Xử lý trạng thái và sinh cảnh báo	17
4.6	Lưu trữ dữ liệu sau xử lý	17
4.7	Chuẩn bị dữ liệu cho dashboard	18
5	Mô hình AI và quy trình huấn luyện	18
5.1	Tổng quan mô hình dự đoán Sepsis	18
5.2	Đặc điểm của dataset	18
5.3	Xử lý dữ liệu huấn luyện	20
5.4	Tạo đặc trưng cho mô hình	21
5.5	Huấn luyện và đánh giá mô hình	22
5.6	Mô hình export sẵn từ Kaggle	23
5.7	Quy trình huấn luyện lại với Airflow và MLflow	23

6	Triển khai và vận hành hệ thống	24
6.1	Môi trường triển khai	24
6.2	Quy trình khởi chạy hệ thống	24
6.3	Theo dõi các dashboard trong hệ thống	25
6.4	Vận hành dashboard và demo hệ thống	25
6.5	Dừng và reset hệ thống	27
7	Kết quả và đánh giá	28
7.1	Kết quả xây dựng hệ thống	28
7.2	Kết quả mô hình dự đoán Sepsis	28
7.3	Kết quả demo trên dashboard	29
7.4	Đánh giá ưu điểm và hạn chế	29
8	Kết luận và hướng phát triển	30
8.1	Kết luận	30
8.2	Hướng phát triển	30
	Phân công nhiệm vụ	32
	Tài liệu tham khảo	33

Danh sách hình vẽ

1	Pipeline xử lý dữ liệu thời gian thực	7
2	Pipeline phục vụ dữ liệu và giám sát	8
3	Apache NiFi	10
4	Apache Kafka	11
5	Apache Spark Streaming	11
6	Apache Cassandra	12
7	HDFS	12
8	Flask API	13
9	Apache Airflow	13
10	MLflow	14
11	Docker Compose	14
12	Ma trận tương quan giữa các đặc trưng trong dataset	20
13	Giao diện dashboard theo dõi dấu hiệu sinh tồn và huyết áp	26
14	Giao diện dashboard hiển thị xác suất dự đoán Sepsis	27

1 Giới thiệu

1.1 Bối cảnh

Trong môi trường hồi sức tích cực, bệnh nhân thường cần được theo dõi liên tục vì tình trạng sức khỏe có thể thay đổi rất nhanh. Các chỉ số như nhịp tim, huyết áp, nhiệt độ, nhịp thở, nồng độ oxy trong máu và nhiều kết quả xét nghiệm khác đều có thể phản ánh nguy cơ bệnh nhân đang chuyển biến xấu. Nếu các dấu hiệu bất thường được phát hiện sớm, đội ngũ y tế có thêm thời gian để kiểm tra và can thiệp kịp thời.

Nhiễm khuẩn huyết, hay Sepsis, là một trong những tình trạng nguy hiểm trong điều trị hồi sức. Vấn đề khó nằm ở chỗ các dấu hiệu ban đầu của Sepsis không phải lúc nào cũng rõ ràng. Chúng có thể xuất hiện rải rác trên nhiều chỉ số khác nhau và thay đổi theo thời gian. Vì vậy, việc chỉ quan sát thủ công từng chỉ số riêng lẻ có thể khiến quá trình phát hiện nguy cơ bị chậm trễ.

Trong thực tế, dữ liệu bệnh nhân trong ICU thường là dữ liệu chuỗi thời gian, được cập nhật liên tục theo từng thời điểm. Khi số lượng bệnh nhân và số lượng chỉ số cần theo dõi tăng lên, hệ thống xử lý dữ liệu thông thường sẽ khó đáp ứng tốt yêu cầu về tốc độ, lưu trữ và trực quan hóa. Đây là lý do các công nghệ Big Data và xử lý dữ liệu thời gian thực phù hợp với bài toán giám sát bệnh nhân và cảnh báo sớm nguy cơ Sepsis.

1.2 Mục tiêu

Trong dự án này, nhóm đặt mục tiêu xây dựng một hệ thống Big Data có khả năng mô phỏng luồng dữ liệu bệnh nhân ICU, xử lý dữ liệu gần thời gian thực và hỗ trợ cảnh báo sớm nguy cơ Sepsis bằng mô hình học máy. Hệ thống không chỉ dừng lại ở việc huấn luyện mô hình dự đoán, mà còn hướng đến việc xây dựng một quy trình hoàn chỉnh từ dữ liệu đầu vào đến giao diện hiển thị kết quả.

**Dữ liệu PSV → Apache NiFi → Apache Kafka → Spark Streaming +
XGBoost → Cassandra/HDFS → Flask API → Dashboard**

Với luồng xử lý trên, nhóm tập trung vào việc đưa dữ liệu .psv từ thư mục dữ liệu vào hệ thống, chuyển đổi dữ liệu thô sang dạng có cấu trúc, xử lý theo từng bệnh nhân và tạo đầu vào phù hợp cho mô hình dự đoán. Sau khi Spark Streaming tính toán xác suất Sepsis, kết quả được lưu vào Cassandra để truy vấn nhanh và HDFS để lưu dữ liệu lịch sử.

Bên cạnh phần xử lý dữ liệu, nhóm cũng xây dựng dashboard để người dùng có thể theo dõi các chỉ số ICU, xác suất Sepsis và trạng thái cảnh báo theo từng bệnh nhân. Ngoài ra, dự án có tích hợp Airflow và MLflow nhằm hỗ trợ quy trình huấn luyện lại mô hình, theo dõi metric và quản lý các artifact sinh ra trong quá trình huấn luyện.

1.3 Tổng quan giải pháp

Giải pháp của dự án được xây dựng theo hướng một hệ thống Big Data nhiều thành phần. Mã nguồn của hệ thống được nhóm quản lý tại [BIGDATA_FINAL](#).

Dữ liệu đầu vào là các file .psv, trong đó mỗi file đại diện cho một bệnh nhân và mỗi dòng thể hiện trạng thái của bệnh nhân tại một thời điểm trong ICU. Apache NiFi được

sử dụng để đọc dữ liệu, trích xuất mã bệnh nhân, chuyển đổi dữ liệu từ định dạng PSV sang JSON và gửi vào Apache Kafka.

Kafka đóng vai trò là tầng trung gian để lưu và truyền luồng dữ liệu. Nhờ Kafka, thành phần thu thập dữ liệu và thành phần xử lý dữ liệu không bị phụ thuộc trực tiếp vào nhau. Apache Spark Structured Streaming sau đó đọc dữ liệu từ Kafka, xử lý theo từng bệnh nhân, thực hiện tiền xử lý dữ liệu và sử dụng mô hình XGBoost để dự đoán xác suất Sepsis.

Kết quả dự đoán được lưu vào hai nơi. Cassandra được dùng để lưu dữ liệu phục vụ truy vấn nhanh cho dashboard. HDFS lưu dữ liệu lịch sử dưới dạng Parquet để phục vụ lưu trữ lâu dài và có thể dùng cho các bước phân tích hoặc huấn luyện lại mô hình. Phía người dùng, Flask API cung cấp các endpoint để dashboard lấy dữ liệu. Nginx được dùng làm tầng reverse proxy và cân bằng tải cho API.

Dashboard được xây dựng bằng Highcharts để hiển thị các chỉ số sinh tồn, chỉ số xét nghiệm, xác suất Sepsis và trạng thái cảnh báo. Người dùng có thể chọn bệnh nhân, xem dữ liệu theo thời gian và theo dõi cảnh báo trực quan. Bên cạnh pipeline thời gian thực, hệ thống còn có Airflow và MLflow để hỗ trợ quy trình huấn luyện lại mô hình, quản lý metric và lưu trữ artifact.

Nhìn tổng thể, hệ thống tạo thành một quy trình khép kín: dữ liệu bệnh nhân được đưa vào hệ thống, xử lý theo luồng, dự đoán nguy cơ Sepsis, lưu trữ kết quả và hiển thị lên dashboard để hỗ trợ theo dõi bệnh nhân.

2 Dữ liệu và bài toán Sepsis

2.1 Nguồn dữ liệu

Dữ liệu sử dụng trong dự án được lấy từ bộ dữ liệu của *PhysioNet/CinC Challenge 2019*. Đây là bộ dữ liệu được xây dựng cho bài toán dự đoán sớm Sepsis từ dữ liệu lâm sàng của bệnh nhân trong khoa hồi sức tích cực. Dữ liệu công khai gồm hai phần chính là **Training Set A** và **Training Set B**. Hai tập này chứa dữ liệu của nhiều bệnh nhân ICU, được ghi nhận theo thời gian và dùng để phát triển các mô hình cảnh báo sớm Sepsis.

Hai tập dữ liệu được nhóm tải từ các đường dẫn sau:

- **Training Set A:** https://physionet.org/content/challenge-2019/1.0.0/training/training_setA/#files-panel
- **Training Set B:** https://physionet.org/content/challenge-2019/1.0.0/training/training_setB/#files-panel

Sau khi tải về, nhóm sử dụng dữ liệu từ cả Training Set A và Training Set B để phục vụ hai mục đích trong hệ thống. Một phần dữ liệu được dùng cho huấn luyện và huấn luyện lại mô hình. Phần còn lại được dùng để mô phỏng luồng dữ liệu bệnh nhân đi vào hệ thống theo thời gian thực. Cách tổ chức này giúp dự án vừa có dữ liệu cho mô hình học máy, vừa có dữ liệu để chạy thử pipeline Big Data từ đầu vào đến dashboard.

2.2 Cấu trúc và đặc điểm dữ liệu

Mỗi bệnh nhân trong bộ dữ liệu được lưu thành một file riêng có định dạng `.psv`. Đây là dạng file văn bản, trong đó các cột được phân tách bằng ký tự `|`. Mỗi dòng trong file biểu diễn thông tin của bệnh nhân tại một thời điểm trong ICU, thường tương ứng với từng giờ nằm viện.

Các file dữ liệu có 41 cột, bao gồm nhiều nhóm thông tin khác nhau. Một số nhóm dữ liệu quan trọng gồm:

- **Dấu hiệu sinh tồn:** HR, O2Sat, Temp, SBP, MAP, DBP, Resp. Các chỉ số này phản ánh trạng thái cơ bản của bệnh nhân như nhịp tim, oxy máu, thân nhiệt, huyết áp và nhịp thở.
- **Chỉ số xét nghiệm:** Lactate, WBC, Platelets, Creatinine, BUN, pH, Glucose. Đây là các chỉ số hỗ trợ đánh giá tình trạng nhiễm trùng, chức năng thận, chuyển hóa và phản ứng của cơ thể.
- **Thông tin nền và thời gian:** Age, Gender, Unit1, Unit2, HospAdmTime, ICULOS. Nhóm này cho biết đặc điểm bệnh nhân và thời gian theo dõi trong ICU.
- **Nhãn dự đoán:** SepsisLabel. Đây là nhãn cho biết trạng thái Sepsis tại từng thời điểm.

Trong dữ liệu có nhiều giá trị bị thiếu, đặc biệt là ở các chỉ số xét nghiệm vì không phải xét nghiệm nào cũng được thực hiện ở mọi thời điểm. Đây là đặc điểm thường gặp của dữ liệu y tế thực tế. Vì vậy, trước khi đưa dữ liệu vào mô hình dự đoán, hệ thống cần có bước tiền xử lý để xử lý giá trị thiếu và chuẩn hóa dữ liệu.

Cột cuối cùng của dữ liệu là `SepsisLabel`. Đây là nhãn cho biết tại từng thời điểm bệnh nhân đã được xem là có nguy cơ Sepsis theo định nghĩa của bộ dữ liệu hay chưa. Với bệnh nhân có Sepsis, nhãn sẽ chuyển sang giá trị 1 từ khoảng thời gian cảnh báo sớm trước thời điểm Sepsis. Với bệnh nhân không có Sepsis, nhãn giữ giá trị 0 trong toàn bộ quá trình theo dõi.

2.3 Tổ chức dữ liệu trong dự án

Trong thư mục dự án, dữ liệu được tổ chức lại theo mục đích sử dụng thay vì giữ nguyên hoàn toàn theo tên Training Set A và Training Set B. Thư mục `train_data` chứa dữ liệu dùng cho quá trình huấn luyện và huấn luyện lại mô hình. Trong phiên bản dữ liệu hiện tại, thư mục này có 30,336 file `.psv`. Đây là nguồn dữ liệu được sử dụng bởi quy trình MLOps, trong đó Airflow gọi script huấn luyện tại `mlops/train.py` để train lại mô hình khi cần.

Thư mục `live_data` chứa dữ liệu dùng để mô phỏng luồng dữ liệu thời gian thực. Trong phiên bản hiện tại, thư mục này có 10,008 file `.psv`. Apache NiFi sẽ quét dữ liệu trong thư mục này, đọc từng file bệnh nhân, trích xuất mã bệnh nhân từ tên file, chuyển dữ liệu sang JSON và gửi vào Kafka. Từ đó, Spark Streaming tiếp tục xử lý dữ liệu, dự đoán nguy cơ Sepsis và ghi kết quả cho dashboard.

Việc chia dữ liệu thành hai phần như trên giúp hệ thống tách rõ hai nhiệm vụ. Phần `train_data` phục vụ huấn luyện mô hình và đánh giá chất lượng mô hình. Phần `live_data` phục vụ trình diễn pipeline thời gian thực, giúp kiểm tra toàn bộ luồng từ dữ liệu đầu vào, xử lý streaming, lưu trữ đến trực quan hóa kết quả.

2.4 Bài toán dự đoán Sepsis

Trong dự án này, bài toán Sepsis được xem như một bài toán dự đoán theo chuỗi thời gian. Đầu vào của bài toán là chuỗi các chỉ số ICU của từng bệnh nhân theo từng giờ. Từ các chỉ số này, hệ thống cần ước lượng xác suất bệnh nhân có nguy cơ Sepsis và đưa ra cảnh báo khi xác suất vượt qua ngưỡng được chọn.

Kết quả đầu ra của mô hình không chỉ là nhãn 0 hoặc 1, mà còn có xác suất dự đoán **SepsisProb**. Giá trị này giúp dashboard hiển thị mức độ rủi ro theo thời gian, thay vì chỉ hiển thị kết luận cuối cùng. Dựa trên xác suất này, hệ thống sinh thêm các trạng thái cảnh báo như **SepsisWarning** và **SepsisConfirmed** để hỗ trợ người dùng theo dõi tình trạng bệnh nhân.

Mục tiêu của bài toán không phải thay thế quyết định của bác sĩ, mà là cung cấp thêm một công cụ hỗ trợ theo dõi. Khi hệ thống phát hiện bệnh nhân có dấu hiệu rủi ro tăng dần, dashboard có thể hiển thị cảnh báo sớm để đội ngũ y tế chú ý và kiểm tra lại tình trạng bệnh nhân kịp thời.

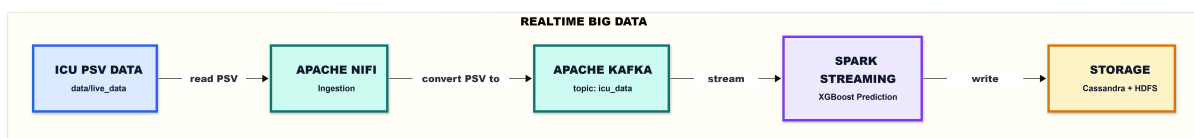
3 Phân tích và thiết kế hệ thống

3.1 Kiến trúc tổng thể

Hệ thống được thiết kế theo hướng pipeline Big Data nhiều tầng, trong đó mỗi thành phần đảm nhiệm một nhiệm vụ rõ ràng: thu thập dữ liệu, truyền dữ liệu, xử lý streaming, lưu trữ, cung cấp API và trực quan hóa kết quả. Cách thiết kế này giúp hệ thống dễ mở rộng hơn so với việc viết toàn bộ xử lý trong một chương trình duy nhất, đồng thời phù hợp với đặc điểm của dữ liệu ICU là dữ liệu chuỗi thời gian và có thể phát sinh liên tục.

Dữ liệu ban đầu được tải từ hai tập *Training Set A* và *Training Set B*, sau đó được tổ chức lại thành các thư mục phục vụ từng mục đích. Thư mục **train_data** dùng cho huấn luyện và huấn luyện lại mô hình, còn **live_data** dùng để mô phỏng dữ liệu realtime đi vào pipeline. Toàn bộ các dịch vụ được điều phối bằng **docker-compose.yml**, bao gồm NiFi, Kafka, Spark, Cassandra, HDFS, Flask API, Nginx, Airflow và MLflow.

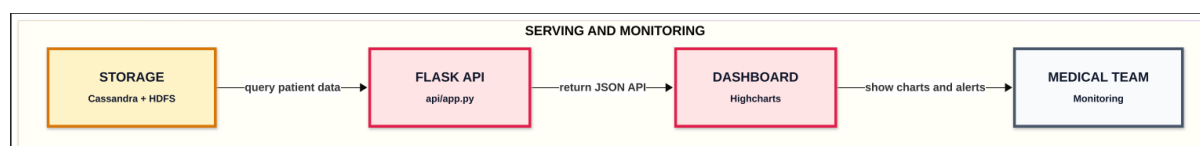
Về mặt thiết kế, hệ thống có thể chia thành hai luồng chính. Luồng thứ nhất là luồng xử lý realtime, nhận dữ liệu PSV, chuyển đổi sang JSON, đưa vào Kafka, xử lý bằng Spark Streaming và ghi kết quả ra storage. Luồng thứ hai là luồng serving và monitoring, lấy dữ liệu đã xử lý từ storage, trả về cho dashboard thông qua API và hiển thị cảnh báo cho người dùng. Ngoài hai luồng này, hệ thống còn có nhánh MLOps gồm Airflow và MLflow để hỗ trợ huấn luyện lại mô hình và quản lý kết quả huấn luyện.



Hình 1: Pipeline xử lý dữ liệu thời gian thực

Hình 1 mô tả luồng xử lý realtime của hệ thống. Dữ liệu bệnh nhân dạng PSV được Apache NiFi đọc từ thư mục **live_data**. NiFi trích xuất mã bệnh nhân từ tên file, chuyển dữ liệu sang JSON và gửi vào Kafka topic **icu_data**. Spark Streaming đọc dữ liệu từ

topic này, xử lý theo từng bệnh nhân, nạp mô hình XGBoost để dự đoán xác suất Sepsis, sau đó ghi kết quả vào Cassandra và HDFS.



Hình 2: Pipeline phục vụ dữ liệu và giám sát

Hình 2 mô tả luồng phục vụ dữ liệu cho người dùng. Flask API ưu tiên đọc dữ liệu từ Cassandra để hiển thị dữ liệu mới nhất cho dashboard. Nếu Cassandra không có dữ liệu, API có thể đọc dữ liệu lịch sử từ HDFS hoặc fallback về file PSV cục bộ để phục vụ demo. Dashboard sử dụng Highcharts để hiển thị các chỉ số ICU, xác suất Sepsis và trạng thái cảnh báo. Trong triển khai bằng Docker Compose, API được chạy với 2 replica và đặt sau Nginx ở cổng 5000.

3.2 Pipeline xử lý dữ liệu thời gian thực

Pipeline xử lý dữ liệu thời gian thực là phần lõi của hệ thống. Mục tiêu của pipeline này là biến dữ liệu bệnh nhân từ các file PSV thành một luồng dữ liệu có thể xử lý liên tục. Dữ liệu sau khi đi qua pipeline không chỉ được dùng để dự đoán nguy cơ Sepsis, mà còn được lưu lại để phục vụ truy vấn, trực quan hóa và phân tích lịch sử.

Ở đầu pipeline, mỗi file PSV đại diện cho một bệnh nhân ICU. Mỗi dòng trong file là dữ liệu tại một thời điểm theo giờ, bao gồm các chỉ số sinh tồn, xét nghiệm, thông tin bệnh nhân và nhãn Sepsis. Tên file được sử dụng để xác định mã bệnh nhân, từ đó giúp hệ thống gom dữ liệu, xử lý trạng thái và truy vấn kết quả theo từng bệnh nhân.

Tầng dữ liệu đầu vào đóng vai trò cung cấp dữ liệu lâm sàng ban đầu cho hệ thống. Dữ liệu ở tầng này là dữ liệu dạng bảng, nhưng chưa phù hợp để truyền trực tiếp trong pipeline streaming. Vì vậy, trước khi đưa vào Kafka, dữ liệu cần được đọc, chuẩn hóa định dạng và bổ sung thông tin định danh bệnh nhân.

Tầng thu thập dữ liệu bằng Apache NiFi chịu trách nhiệm tự động quét các file PSV, đọc nội dung file, lấy mã bệnh nhân từ tên file và chuyển đổi dữ liệu sang dạng JSON. Đây là bước quan trọng vì Kafka và Spark xử lý dữ liệu dạng bản ghi tốt hơn so với file PSV thô. Ngoài nhánh gửi dữ liệu sang Kafka, NiFi còn có thể sao lưu dữ liệu thô lên HDFS để phục vụ lưu trữ và kiểm tra lại khi cần.

Tầng truyền dữ liệu bằng Apache Kafka đóng vai trò là hàng đợi trung gian giữa NiFi và Spark Streaming. NiFi có thể đẩy dữ liệu vào Kafka, còn Spark đọc dữ liệu từ Kafka theo tốc độ xử lý riêng. Nhờ đó, hai thành phần này không phụ thuộc trực tiếp vào nhau. Nếu dữ liệu được đưa vào nhanh hơn tốc độ xử lý của Spark, Kafka vẫn có thể giữ dữ liệu trong topic để Spark tiếp tục đọc sau. Điều này giúp pipeline ổn định hơn khi mô phỏng nhiều bệnh nhân hoặc khi dữ liệu vào liên tục.

Tầng xử lý bằng Spark Streaming nhận dữ liệu từ Kafka và thực hiện các bước xử lý chính. Dữ liệu JSON được chuyển về đúng kiểu dữ liệu, bổ sung thời điểm xử lý và gom theo từng bệnh nhân. Vì bài toán Sepsis phụ thuộc vào diễn biến theo thời gian, Spark không xử lý từng dòng một cách độc lập, mà giữ trạng thái của từng bệnh nhân để theo dõi chuỗi chỉ số qua nhiều giờ. Từ chuỗi dữ liệu này, hệ thống tạo đầu vào cho mô hình XGBoost để tính xác suất nguy cơ Sepsis.

Kết quả sau xử lý gồm xác suất dự đoán Sepsis và các trạng thái cảnh báo. Trong hệ thống, xác suất dự đoán được biểu diễn bằng `SepsisProb`. Từ giá trị này, hệ thống sinh thêm hai trạng thái `SepsisWarning` và `SepsisConfirmed`. `SepsisWarning` thể hiện bệnh nhân đã bắt đầu có dấu hiệu rủi ro, còn `SepsisConfirmed` thể hiện mức cảnh báo cao hơn khi rủi ro kéo dài hoặc vượt điều kiện xác nhận.

Tầng lưu trữ nhận kết quả từ Spark và ghi ra hai nơi. Cassandra được dùng để lưu dữ liệu mới, phục vụ truy vấn nhanh cho dashboard. HDFS được dùng để lưu dữ liệu lịch sử ở dạng Parquet, phù hợp cho lưu trữ dài hạn và huấn luyện lại mô hình. Cách ghi song song này giúp hệ thống vừa đáp ứng nhu cầu realtime, vừa giữ lại dữ liệu phục vụ phân tích sau này.

Điểm quan trọng của pipeline này là các tầng được tách nhiệm vụ rõ ràng. NiFi tập trung vào thu thập và chuyển đổi dữ liệu, Kafka tập trung vào truyền dữ liệu, Spark tập trung vào xử lý và dự đoán, còn Cassandra và HDFS tập trung vào lưu trữ. Nhờ cách thiết kế này, hệ thống dễ theo dõi, dễ mở rộng và phù hợp với bài toán xử lý dữ liệu y tế theo thời gian thực.

3.3 Pipeline phục vụ dữ liệu và giám sát

Pipeline phục vụ dữ liệu và giám sát là phần đưa kết quả xử lý đến người dùng cuối. Nếu pipeline realtime trả lời câu hỏi dữ liệu được xử lý như thế nào, thì pipeline serving trả lời câu hỏi dữ liệu được truy vấn và hiển thị ra sao. Trong dự án, phần này được hiện thực chủ yếu bằng Flask API, Nginx và dashboard Highcharts.

Dữ liệu đầu vào của pipeline này là dữ liệu đã được Spark ghi vào Cassandra và HDFS. Cassandra được xem là nguồn đọc chính vì phù hợp với truy vấn nhanh theo bệnh nhân và thời gian ICU. HDFS đóng vai trò lưu trữ lịch sử và nguồn dữ liệu dự phòng. Ngoài ra, khi chạy demo hoặc khi hệ thống lưu trữ chưa có dữ liệu, API vẫn có thể đọc trực tiếp từ file PSV cục bộ để dashboard không bị trống.

Tầng lưu trữ phục vụ truy vấn gồm Cassandra và HDFS. Cassandra lưu dữ liệu dạng chuỗi thời gian theo từng bệnh nhân, gồm các chỉ số ICU, nhãn thật, xác suất dự đoán và trạng thái cảnh báo. Dữ liệu trong Cassandra được tối ưu cho truy vấn nhanh nên phù hợp với dashboard realtime. Trong khi đó, HDFS lưu dữ liệu lịch sử dạng Parquet, phù hợp cho các tác vụ cần đọc lại dữ liệu theo lô hoặc phục vụ huấn luyện lại mô hình.

Tầng API đóng vai trò trung gian giữa dữ liệu lưu trữ và giao diện người dùng. Dashboard không truy cập trực tiếp vào Cassandra hay HDFS, mà gửi yêu cầu đến API. API nhận mã bệnh nhân, truy vấn dữ liệu tương ứng, chuẩn hóa kết quả thành JSON và trả về cho dashboard. Cách thiết kế này giúp tách giao diện khỏi logic truy vấn dữ liệu, đồng thời giúp hệ thống dễ thay đổi tầng lưu trữ mà không cần sửa nhiều ở phía dashboard.

Một điểm quan trọng của tầng API là cơ chế fallback. Khi có yêu cầu dữ liệu của một bệnh nhân, API ưu tiên đọc từ Cassandra để lấy dữ liệu mới nhất. Nếu Cassandra không có dữ liệu, API chuyển sang đọc dữ liệu lịch sử từ HDFS. Nếu HDFS cũng chưa có dữ liệu, API có thể đọc trực tiếp từ file PSV cục bộ để phục vụ chế độ demo. Nhờ đó, dashboard vẫn có dữ liệu hiển thị trong nhiều tình huống khác nhau, kể cả khi pipeline realtime chưa ghi dữ liệu vào storage.

Tầng reverse proxy bằng Nginx giúp người dùng truy cập hệ thống qua một địa chỉ thống nhất. Nginx nhận request từ trình duyệt và chuyển tiếp đến các API server phía sau. Trong hệ thống, API có thể chạy nhiều replica để chia tải truy vấn. Việc đặt

API sau Nginx giúp kiến trúc serving linh hoạt hơn và phù hợp hơn với hướng triển khai thực tế.

Tầng dashboard là nơi người dùng tương tác trực tiếp với hệ thống. Dashboard hiển thị dữ liệu dưới dạng biểu đồ theo thời gian, giúp người dùng theo dõi diễn biến của từng bệnh nhân. Các nhóm biểu đồ gồm dấu hiệu sinh tồn, huyết áp, chỉ số xét nghiệm, chỉ số hóa sinh và xác suất Sepsis. Bên cạnh đó, dashboard còn hiển thị trạng thái cảnh báo để người dùng nhận biết nhanh bệnh nhân đang bình thường, có cảnh báo sớm hay có nguy cơ cao.

Đội ngũ y tế là đối tượng sử dụng kết quả cuối cùng của hệ thống. Kết quả dự đoán không thay thế kết luận chuyên môn, nhưng đóng vai trò như một công cụ hỗ trợ theo dõi. Khi xác suất Sepsis tăng hoặc hệ thống sinh cảnh báo, đội ngũ y tế có thể kiểm tra lại tình trạng bệnh nhân và đưa ra quyết định phù hợp.

Thiết kế fallback của API là một điểm quan trọng trong hệ thống. Nếu chỉ đọc Cassandra, dashboard có thể bị trống khi Cassandra chưa có dữ liệu hoặc dữ liệu đã hết TTL. Vì vậy, API được thiết kế theo thứ tự ưu tiên: Cassandra cho dữ liệu realtime, HDFS cho dữ liệu lịch sử, và file PSV local cho tình huống demo. Cách này giúp giao diện vẫn hoạt động ổn định trong nhiều trạng thái khác nhau của hệ thống.

3.4 Công nghệ sử dụng

3.4.1 Apache NiFi

Apache NiFi là công cụ dùng để xây dựng luồng thu thập và chuyển đổi dữ liệu. NiFi phù hợp với bài toán cần đọc dữ liệu từ file, xử lý theo từng bước và đẩy dữ liệu sang hệ thống khác.



Hình 3: Apache NiFi

Trong dự án, NiFi đọc dữ liệu từ `data/live_data`, lấy mã bệnh nhân từ tên file, chuyển dữ liệu PSV sang JSON và gửi vào Kafka. Việc sử dụng NiFi giúp phân thu thập dữ liệu được tách riêng khỏi phần xử lý streaming, đồng thời dễ quan sát và điều chỉnh pipeline khi cần.

3.4.2 Apache Kafka

Apache Kafka là hệ thống truyền dữ liệu dạng luồng, thường được dùng làm message broker trong các hệ thống realtime. Kafka giúp lưu tạm và truyền dữ liệu giữa các thành phần mà không làm chúng phụ thuộc trực tiếp vào nhau.



Hình 4: Apache Kafka

Trong hệ thống này, Kafka nhận dữ liệu JSON từ NiFi qua topic `icu_data`. Spark Streaming sau đó đọc dữ liệu từ topic này để xử lý. Kafka giúp hệ thống chịu được trường hợp dữ liệu vào nhanh hơn tốc độ xử lý, vì dữ liệu có thể được lưu trong topic trước khi Spark tiêu thụ.

3.4.3 Apache Spark Streaming

Apache Spark Streaming là thành phần xử lý dữ liệu theo luồng. Spark có khả năng xử lý dữ liệu lớn, hỗ trợ xử lý song song và tích hợp tốt với Kafka, Cassandra và HDFS.



Hình 5: Apache Spark Streaming

Trong dự án, Spark Streaming đọc dữ liệu từ Kafka, parse dữ liệu JSON, xử lý theo từng `patient_id` và sử dụng mô hình XGBoost để dự đoán xác suất Sepsis. Kết quả được ghi đồng thời vào Cassandra và HDFS, giúp vừa phục vụ dashboard vừa lưu lại dữ liệu lịch sử.

3.4.4 Apache Cassandra

Apache Cassandra là cơ sở dữ liệu NoSQL phân tán, phù hợp với dữ liệu ghi liên tục và truy vấn nhanh theo khóa. Cassandra thường được sử dụng trong các hệ thống cần lưu dữ liệu thời gian thực với khả năng mở rộng cao.



Hình 6: Apache Cassandra

Trong dự án, Cassandra lưu bảng `icu_readings` trong keyspace `sepsis_monitoring`. Bảng này lưu dữ liệu theo `patient_id` và ICULOS, phù hợp với cách dashboard truy vấn dữ liệu của từng bệnh nhân theo thời gian.

3.4.5 HDFS

HDFS là hệ thống lưu trữ phân tán trong hệ sinh thái Hadoop. HDFS phù hợp để lưu dữ liệu dung lượng lớn và phục vụ các bài toán xử lý dữ liệu lịch sử.



Hình 7: HDFS

Trong hệ thống, HDFS được dùng để lưu dữ liệu Parquet tại `/data/processed_parquet` và lưu checkpoint cho Spark Streaming. HDFS cũng đóng vai trò nguồn dữ liệu fallback cho API khi dữ liệu trong Cassandra không còn hoặc chưa truy vấn được.

3.4.6 Flask API

Flask là framework web nhẹ của Python, phù hợp để xây dựng các API nhanh, gọn và dễ triển khai. Với thiết kế đơn giản, Flask thường được dùng trong các hệ thống cần một lớp trung gian để nhận request, xử lý logic và trả dữ liệu về cho phía giao diện.



Hình 8: Flask API

Trong dự án, Flask API đóng vai trò là lớp trung gian giữa tầng lưu trữ và dashboard. API nhận yêu cầu truy vấn theo bệnh nhân, đọc dữ liệu từ Cassandra hoặc HDFS, chuẩn hóa kết quả và trả về dữ liệu dạng JSON. Nhờ có API, dashboard không cần truy cập trực tiếp vào cơ sở dữ liệu, giúp hệ thống dễ bảo trì và dễ thay đổi tầng lưu trữ phía sau.

3.4.7 Apache Airflow

Apache Airflow là công cụ dùng để lập lịch và tự động hóa các workflow. Airflow cho phép định nghĩa các quy trình xử lý theo dạng DAG, trong đó mỗi task thể hiện một bước cần thực hiện. Công cụ này phù hợp với các hệ thống cần chạy tác vụ định kỳ, ví dụ như kiểm tra dữ liệu mới hoặc huấn luyện lại mô hình.



Hình 9: Apache Airflow

Trong dự án, Airflow được dùng để lập lịch quy trình huấn luyện lại mô hình. Khi đến thời điểm chạy, Airflow kích hoạt task huấn luyện, sử dụng dữ liệu mới và dữ liệu lịch sử để cập nhật mô hình. Việc sử dụng Airflow giúp quá trình huấn luyện không cần thực hiện thủ công, đồng thời dễ theo dõi trạng thái thành công hoặc lỗi của từng lần chạy.

3.4.8 MLflow

MLflow là nền tảng hỗ trợ quản lý vòng đời mô hình học máy. MLflow có thể lưu lại tham số huấn luyện, metric đánh giá, artifact và phiên bản mô hình. Nhờ đó, nhóm có thể so sánh các lần huấn luyện khác nhau và kiểm soát tốt hơn quá trình cập nhật mô hình.

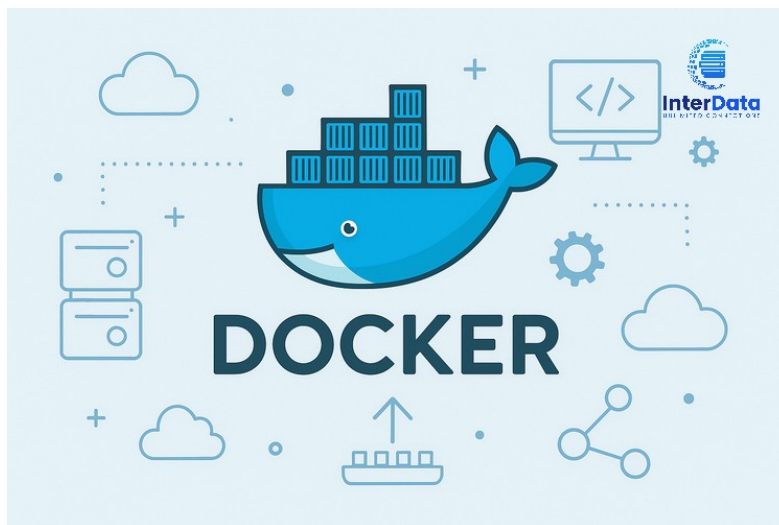


Hình 10: MLflow

Trong dự án, MLflow được dùng để ghi lại thông tin trong quá trình huấn luyện XGBoost, bao gồm metric đánh giá, model artifact, scaler và các kết quả liên quan. Việc lưu vết bằng MLflow giúp nhóm biết mô hình nào đang hoạt động tốt hơn, đồng thời hỗ trợ quản lý phiên bản mô hình khi hệ thống cần huấn luyện lại.

3.4.9 Docker Compose

Docker Compose được sử dụng để đóng gói và chạy toàn bộ hệ thống dưới dạng nhiều container. Cách triển khai này giúp các thành phần như Kafka, NiFi, Spark, Cassandra, HDFS, API, Nginx, Airflow và MLflow có thể khởi động cùng nhau bằng một file cấu hình.



Hình 11: Docker Compose

Trong dự án, file `docker-compose.yml` là điểm khởi chạy chính. File này khai báo các service, volume, network, port và dependency giữa các thành phần. Nhờ Docker Compose, nhóm có thể triển khai hệ thống nhất quán hơn trên môi trường local hoặc khi demo.

4 Quy trình xử lý và phân tích dữ liệu

4.1 Tổng quan luồng xử lý dữ liệu

Quy trình xử lý dữ liệu của hệ thống bắt đầu từ dữ liệu bệnh nhân ICU dạng PSV và kết thúc ở dữ liệu đã sẵn sàng để hiển thị trên dashboard. Dữ liệu ban đầu là dữ liệu theo thời gian, trong đó mỗi bệnh nhân có một chuỗi bản ghi theo từng giờ nằm ICU. Vì vậy, hệ thống không xử lý dữ liệu như các dòng độc lập, mà cần giữ được quan hệ theo thời gian của từng bệnh nhân.

Luồng xử lý chính của hệ thống có thể tóm tắt như sau:

**PSV → JSON → Kafka Stream → Spark Streaming → Feature Engineering
→ Sepsis Prediction → Cassandra/HDFS**

Trong luồng này, Apache NiFi chịu trách nhiệm đọc và chuyển đổi dữ liệu đầu vào. Apache Kafka đóng vai trò truyền dữ liệu dạng stream. Apache Spark Structured Streaming là nơi thực hiện phần xử lý chính, bao gồm parse dữ liệu, xử lý thiếu dữ liệu, tạo đặc trưng theo chuỗi thời gian, chạy mô hình XGBoost và sinh kết quả cảnh báo. Sau đó, dữ liệu được ghi vào Cassandra để dashboard truy vấn nhanh và HDFS để lưu trữ lịch sử.

Phần này tập trung vào quy trình xử lý dữ liệu trong lúc hệ thống vận hành realtime. Các bước xử lý dữ liệu dùng cho huấn luyện mô hình, cách chia tập train/validation, lựa chọn ngưỡng và đánh giá mô hình sẽ được trình bày riêng ở phần mô hình AI và huấn luyện. Tuy nhiên, hai quy trình này cần thống nhất về cách xử lý thiếu dữ liệu, chuẩn hóa và tạo đặc trưng để mô hình khi huấn luyện và khi chạy realtime nhận cùng một kiểu đầu vào.

4.2 Thu thập và chuyển đổi dữ liệu đầu vào

Dữ liệu đầu vào của hệ thống là các file PSV, trong đó mỗi file đại diện cho một bệnh nhân. Các cột trong file được phân tách bằng ký tự `|`. Đây là định dạng phù hợp để lưu dữ liệu dạng bảng, nhưng chưa thuận tiện để truyền qua hệ thống streaming. Vì vậy, bước đầu tiên là chuyển dữ liệu từ dạng file PSV sang các bản ghi JSON.

Apache NiFi đảm nhiệm bước thu thập và chuyển đổi dữ liệu. NiFi quét các file bệnh nhân, đọc nội dung file và lấy mã bệnh nhân từ tên file. Mã bệnh nhân này được thêm vào dữ liệu dưới trường `patient_id`. Đây là trường rất quan trọng vì các bước xử lý phía sau đều cần biết bản ghi thuộc về bệnh nhân nào.

Sau khi có `patient_id`, NiFi chuyển dữ liệu từ PSV sang JSON. Mỗi bản ghi JSON chứa đầy đủ các chỉ số ICU như nhịp tim, oxy máu, huyết áp, xét nghiệm, thời gian nằm ICU và nhãn `SepsisLabel`. Dữ liệu JSON sau đó được gửi vào Kafka topic `icu_data`. Nhờ bước này, dữ liệu từ dạng file tĩnh được biến thành luồng dữ liệu có thể xử lý liên tục.

Ngoài nhánh gửi dữ liệu vào Kafka, hệ thống còn có nhánh lưu dữ liệu thô vào HDFS. Việc lưu lại dữ liệu thô giúp hệ thống có nguồn dữ liệu dự phòng, đồng thời thuận tiện cho việc kiểm tra hoặc xử lý lại dữ liệu khi cần.

4.3 Tiền xử lý dữ liệu trong Spark Streaming

Spark Streaming đọc dữ liệu từ Kafka topic `icu_data`. Dữ liệu nhận được từ Kafka là chuỗi JSON, nên bước đầu tiên trong Spark là parse JSON về dạng bảng có schema rõ ràng. Các cột dữ liệu được ép về đúng kiểu số, ví dụ các chỉ số sinh tồn và xét nghiệm được chuyển về kiểu số thực, nhãn Sepsis được chuyển về kiểu số nguyên, còn `patient_id` được giữ dưới dạng chuỗi.

Sau khi parse dữ liệu, Spark bổ sung thêm thời điểm xử lý `event_time`. Trường này thể hiện thời điểm hệ thống nhận và xử lý bản ghi, khác với `ICULOS`, là số giờ bệnh nhân nằm ICU trong dữ liệu gốc. Hai trường thời gian này có ý nghĩa khác nhau: `ICULOS` dùng để sắp xếp diễn biến lâm sàng của bệnh nhân, còn `event_time` giúp dashboard biết dữ liệu được hệ thống xử lý vào thời điểm nào.

Một đặc điểm quan trọng của dữ liệu ICU là có nhiều giá trị bị thiếu. Ví dụ, các chỉ số sinh tồn như nhịp tim hoặc huyết áp có thể xuất hiện thường xuyên, nhưng các chỉ số xét nghiệm như Lactate, WBC hoặc Creatinine không phải lúc nào cũng có ở mọi giờ. Vì vậy, hệ thống cần xử lý giá trị thiếu trước khi đưa dữ liệu vào mô hình.

Quy trình xử lý giá trị thiếu được thực hiện theo hai bước. Trước hết, các giá trị thiếu được thay bằng giá trị trung bình thống kê của từng biến. Sau đó, hệ thống áp dụng cơ chế forward-fill theo thời gian, nghĩa là nếu tại một thời điểm một chỉ số bị thiếu, hệ thống sử dụng giá trị gần nhất trước đó của chính bệnh nhân đó. Cách xử lý này phù hợp với dữ liệu chuỗi thời gian y tế, vì nhiều chỉ số thường không thay đổi đột ngột trong khoảng thời gian ngắn.

Bên cạnh dữ liệu đã được điền thiếu, hệ thống còn tạo thêm một mặt nạ dữ liệu. Mặt nạ này cho biết tại từng thời điểm, chỉ số nào là giá trị thật được ghi nhận và chỉ số nào là giá trị được điền. Nhờ đó, mô hình không chỉ biết giá trị của chỉ số, mà còn biết mức độ đầy đủ của dữ liệu tại thời điểm đó.

4.4 Tạo đặc trưng theo chuỗi thời gian

Phần tạo đặc trưng được thực hiện trong luồng xử lý Spark Streaming, sau khi dữ liệu đã được parse, ép kiểu và xử lý giá trị thiếu. Đây là bước quan trọng vì mô hình Sepsis không chỉ cần giá trị hiện tại của bệnh nhân, mà còn cần biết xu hướng thay đổi của các chỉ số trong những giờ gần nhất.

Hệ thống sử dụng 40 biến đầu vào chính, bao gồm các chỉ số sinh tồn, xét nghiệm và thông tin nền của bệnh nhân. Với mỗi thời điểm, hệ thống tạo ba nhóm đặc trưng:

- **Giá trị đã chuẩn hóa:** các chỉ số ICU sau khi xử lý thiếu và chuẩn hóa theo thống kê huấn luyện.
- **Độ thay đổi theo thời gian:** chênh lệch của từng chỉ số so với giờ trước đó, giúp mô hình nhận biết xu hướng tăng hoặc giảm.
- **Mặt nạ dữ liệu:** cho biết chỉ số nào có giá trị gốc và chỉ số nào được điền do thiếu dữ liệu.

Như vậy, với 40 biến ban đầu, mỗi giờ dữ liệu được mở rộng thành 120 đặc trưng, gồm 40 giá trị đã chuẩn hóa, 40 giá trị thay đổi theo thời gian và 40 giá trị mặt nạ. Đây là cách giúp mô hình hiểu dữ liệu sâu hơn thay vì chỉ nhìn vào giá trị đo được tại một thời điểm.

Đối với một số chỉ số xét nghiệm có phân phối lệch mạnh, hệ thống áp dụng biến đổi log trước khi chuẩn hóa. Cách làm này giúp giảm ảnh hưởng của các giá trị quá lớn và làm dữ liệu ổn định hơn cho mô hình. Các biến còn lại được chuẩn hóa bằng trung bình và độ lệch chuẩn đã tính từ dữ liệu huấn luyện.

Sau khi tạo đặc trưng theo từng giờ, hệ thống tiếp tục tạo cửa sổ thời gian gồm 6 giờ gần nhất. Mỗi cửa sổ lấy đặc trưng của thời điểm hiện tại và các thời điểm trước đó. Vì mỗi giờ có 120 đặc trưng, cửa sổ 6 giờ tạo ra tổng cộng 720 đặc trưng cho mô hình XGBoost. Nếu bệnh nhân chưa có đủ 6 giờ dữ liệu, hệ thống thêm các giá trị 0 vào phần còn thiếu để đảm bảo kích thước đầu vào luôn cố định.

Thiết kế đặc trưng dạng cửa sổ giúp mô hình xem xét cả trạng thái hiện tại và diễn biến gần đây của bệnh nhân. Điều này phù hợp với bài toán Sepsis, vì nguy cơ bệnh thường thể hiện qua xu hướng thay đổi liên tục của nhiều chỉ số, không chỉ qua một giá trị đơn lẻ.

4.5 Xử lý trạng thái và sinh cảnh báo

Sau khi tạo đặc trưng, hệ thống dùng mô hình XGBoost để tính xác suất nguy cơ Sepsis tại từng thời điểm. Kết quả dự đoán được lưu trong trường **SepsisProb**. Giá trị này là một số liên tục, thể hiện mức độ rủi ro của bệnh nhân theo đánh giá của mô hình.

Tuy nhiên, nếu chỉ dựa vào một lần dự đoán vượt ngưỡng thì cảnh báo có thể bị nhiễu. Vì vậy, hệ thống sử dụng thêm logic xử lý trạng thái theo từng bệnh nhân. Spark giữ lại lịch sử dữ liệu và trạng thái cảnh báo của mỗi bệnh nhân, sau đó cập nhật trạng thái này khi có bản ghi mới.

Quy trình sinh cảnh báo được thiết kế theo hướng thận trọng hơn so với việc chỉ phân loại 0 hoặc 1. Hệ thống theo dõi các dự đoán gần đây của bệnh nhân. Khi trong các dự đoán gần nhất có đủ số lần dự đoán dương tính, trạng thái **SepsisWarning** được bật để thể hiện cảnh báo sớm. Nếu cảnh báo kéo dài và vẫn có tín hiệu rủi ro trong một khoảng thời gian tiếp theo, trạng thái **SepsisConfirmed** được bật để thể hiện mức cảnh báo cao hơn.

Cách xử lý này giúp cảnh báo ổn định hơn. Một dự đoán dương tính đơn lẻ có thể chỉ là nhiễu, nhưng nhiều tín hiệu rủi ro xuất hiện trong chuỗi thời gian sẽ đáng chú ý hơn. Đây là lý do hệ thống cần xử lý theo trạng thái của từng bệnh nhân thay vì chỉ xử lý từng bản ghi độc lập.

4.6 Lưu trữ dữ liệu sau xử lý

Sau khi Spark xử lý xong dữ liệu và sinh kết quả dự đoán, hệ thống ghi dữ liệu ra hai nơi: Cassandra và HDFS. Đây là thiết kế dual-write, giúp cùng một kết quả có thể phục vụ hai mục đích khác nhau.

Cassandra được dùng cho truy vấn nhanh. Dữ liệu trong Cassandra được tổ chức theo bệnh nhân và thời gian ICU, phù hợp với cách dashboard cần lấy dữ liệu của một bệnh nhân để vẽ biểu đồ theo thời gian. Các trường được lưu gồm các chỉ số ICU, nhãn thật **SepsisLabel**, xác suất dự đoán **SepsisProb**, cảnh báo sớm **SepsisWarning** và cảnh báo xác nhận **SepsisConfirmed**.

HDFS được dùng để lưu dữ liệu lịch sử dưới dạng Parquet. Dữ liệu Parquet có ưu điểm là lưu trữ hiệu quả và phù hợp cho các tác vụ phân tích dữ liệu lớn. Hệ thống phân vùng dữ liệu theo **patient_id**, giúp việc đọc lại dữ liệu theo từng bệnh nhân thuận tiện

hơn. Dữ liệu trên HDFS cũng có thể dùng làm nguồn cho quy trình huấn luyện lại mô hình.

Ngoài dữ liệu kết quả, Spark còn sử dụng checkpoint để lưu trạng thái xử lý streaming. Checkpoint giúp hệ thống có thể tiếp tục xử lý ổn định khi job streaming được chạy lại, hạn chế mất trạng thái của các bệnh nhân đang được theo dõi.

4.7 Chuẩn bị dữ liệu cho dashboard

Dữ liệu sau khi được lưu vào Cassandra và HDFS sẽ được API truy vấn để phục vụ dashboard. Khi người dùng chọn một bệnh nhân, API lấy dữ liệu theo `patient_id`, sắp xếp theo thời gian ICU và chuyển kết quả thành dạng JSON. Dữ liệu JSON này được dashboard sử dụng để vẽ các biểu đồ theo thời gian.

Các nhóm dữ liệu được chuẩn bị cho dashboard gồm chỉ số sinh tồn, huyết áp, xét nghiệm, chỉ số hóa sinh, xác suất Sepsis và trạng thái cảnh báo. Nhờ đó, người dùng có thể quan sát cả dữ liệu lâm sàng gốc lẫn kết quả phân tích của mô hình trên cùng một giao diện.

Trong trường hợp Cassandra chưa có dữ liệu hoặc dữ liệu đã hết thời gian lưu, API có thể đọc dữ liệu lịch sử từ HDFS. Nếu HDFS cũng chưa có dữ liệu, API có thể đọc trực tiếp từ file PSV để phục vụ chế độ demo. Cơ chế này giúp dashboard vẫn hoạt động trong nhiều trạng thái khác nhau của hệ thống, đồng thời hỗ trợ quá trình trình diễn và kiểm thử.

5 Mô hình AI và quy trình huấn luyện

5.1 Tổng quan mô hình dự đoán Sepsis

Trong dự án, mô hình AI được sử dụng để dự đoán nguy cơ Sepsis theo từng thời điểm của bệnh nhân ICU. Đầu vào của mô hình là chuỗi đặc trưng được tạo từ các chỉ số sinh tồn, xét nghiệm và thông tin nền của bệnh nhân trong những giờ gần nhất. Đầu ra của mô hình là xác suất nguy cơ Sepsis, sau đó hệ thống dùng xác suất này để sinh cảnh báo trên dashboard.

Nhóm sử dụng mô hình XGBoost cho bài toán này. XGBoost phù hợp với dữ liệu dạng bảng, có khả năng học các quan hệ phi tuyến giữa nhiều đặc trưng và xử lý tốt dữ liệu có nhiễu. Đây là lựa chọn phù hợp với bộ dữ liệu ICU, vì dữ liệu gồm nhiều nhóm chỉ số khác nhau, có nhiều giá trị thiếu và có sự mất cân bằng lớn giữa lớp Sepsis và không Sepsis.

Trong hệ thống hiện tại, mô hình được sử dụng theo hai hướng. Thứ nhất, hệ thống có sẵn một mô hình đã được huấn luyện và export từ môi trường Kaggle để Spark Streaming có thể nạp trực tiếp khi chạy realtime. Thứ hai, hệ thống có quy trình huấn luyện lại mô hình khi có thêm dữ liệu mới, trong đó Airflow đảm nhiệm lập lịch và MLflow lưu lại metric, artifact và phiên bản mô hình.

5.2 Đặc điểm của dataset

Bộ dữ liệu dùng để huấn luyện được lấy từ PhysioNet/CinC Challenge 2019, gồm hai phần Training Set A và Training Set B. Dữ liệu có dạng chuỗi thời gian, mỗi bệnh nhân

được lưu trong một file PSV riêng. Mỗi dòng dữ liệu tương ứng với một giờ theo dõi trong ICU, gồm các chỉ số sinh tồn, xét nghiệm, thông tin nền và nhãn `SepsisLabel`.

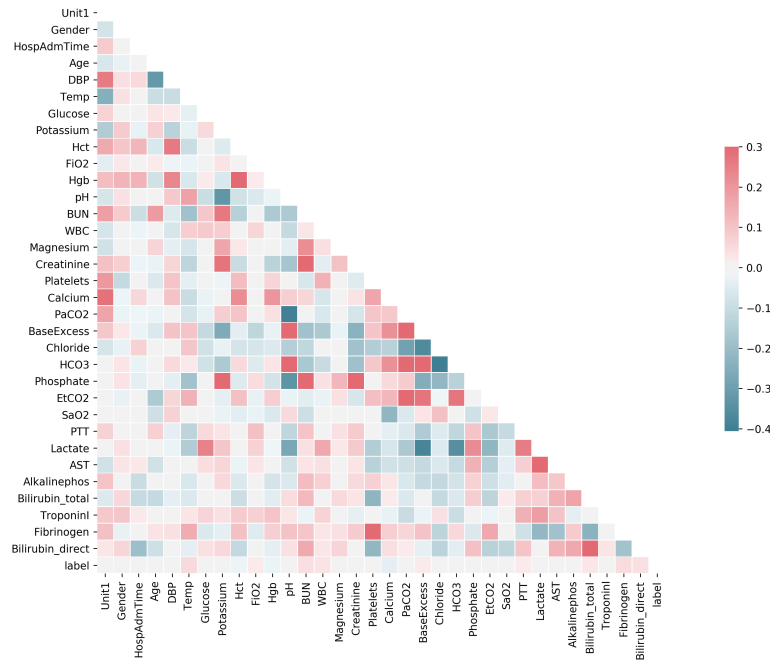
Trong quá trình huấn luyện, nhóm chia dữ liệu theo bệnh nhân thay vì chia theo từng dòng. Cách chia này rất quan trọng vì dữ liệu của cùng một bệnh nhân có quan hệ theo thời gian. Nếu một bệnh nhân xuất hiện đồng thời ở tập train và tập test, mô hình có thể học được đặc điểm riêng của bệnh nhân đó và làm kết quả đánh giá trở nên quá lạc quan.

Đặc điểm	Giá trị
Tổng số bệnh nhân	40,336
Số bệnh nhân Sepsis	2,932
Số bệnh nhân không Sepsis	37,404
Tỷ lệ bệnh nhân Sepsis	7.27%
Tỷ lệ bệnh nhân Sepsis : không Sepsis	1 : 12.76
Số bệnh nhân train / validation / test	28,235 / 6,050 / 6,051
Tổng số time-step sau tạo đặc trưng	1,552,210
Time-step dương <code>SepsisLabel = 1</code>	27,916
Time-step âm <code>SepsisLabel = 0</code>	1,524,294
Tỷ lệ time-step dương	1.80%
Trọng số dương khi huấn luyện	54.54

Bảng 1: Thống kê đặc điểm cơ bản của dataset dùng để huấn luyện

Bảng 1 cho thấy dataset có hai đặc điểm đáng chú ý. Thứ nhất, số bệnh nhân không Sepsis lớn hơn nhiều so với số bệnh nhân Sepsis. Thứ hai, khi xét theo từng time-step, nhãn dương còn hiếm hơn vì Sepsis chỉ xuất hiện trong một khoảng thời gian nhất định của bệnh nhân. Tỷ lệ time-step dương chỉ khoảng 1.80%, nên nếu chỉ dùng accuracy thì mô hình có thể đạt điểm cao bằng cách dự đoán phần lớn là không Sepsis. Vì vậy, khi huấn luyện nhóm sử dụng trọng số dương `scale_pos_weight` để mô hình chú ý hơn đến các mẫu Sepsis.

Ngoài thống kê số lượng, nhóm cũng phân tích tương quan giữa các đặc trưng. Ma trận tương quan giúp quan sát mối quan hệ tuyến tính giữa các chỉ số như huyết áp, xét nghiệm, thông tin bệnh nhân và nhãn Sepsis. Đây là bước phân tích tổng quan để hiểu dữ liệu trước khi huấn luyện mô hình.



Hình 12: Ma trận tương quan giữa các đặc trưng trong dataset

Từ Hình 12, có thể thấy phần lớn hệ số tương quan nằm ở mức vừa phải, không xuất hiện nhiều cặp đặc trưng có tương quan tuyến tính quá mạnh. Vì vậy, nhóm không loại bỏ đặc trưng chỉ dựa trên tương quan. Cách xử lý này phù hợp với XGBoost, vì mô hình cây có thể làm việc tốt với dữ liệu nhiều đặc trưng và tự chọn các đặc trưng hữu ích trong quá trình huấn luyện.

5.3 Xử lý dữ liệu huấn luyện

Quy trình xử lý dữ liệu huấn luyện được xây dựng theo đúng định dạng mà Spark Streaming sẽ sử dụng khi chạy realtime. Đây là điểm quan trọng vì nếu mô hình được train bằng một kiểu dữ liệu nhưng khi triển khai lại nhận đầu vào theo kiểu khác, kết quả dự đoán sẽ không ổn định. Vì vậy, thứ tự đặc trưng, cách xử lý thiếu dữ liệu, cách chuẩn hóa, biến đổi log và kích thước cửa sổ thời gian đều được cố định và lưu lại cùng mô hình.

Đầu tiên, dữ liệu được đọc theo từng bệnh nhân từ hai tập Training Set A và Training Set B. Với mỗi bệnh nhân, hệ thống kiểm tra đủ 40 cột đầu vào theo đúng thứ tự đã định, gồm các chỉ số sinh tồn, xét nghiệm, thông tin nền và thời gian ICU. Nhãn **SepsisLabel** được lấy làm nhãn huấn luyện tại từng time-step. Trước khi train, nhóm xác định bệnh nhân có Sepsis hay không bằng cách kiểm tra xem trong toàn bộ chuỗi thời gian của bệnh nhân có xuất hiện nhãn **SepsisLabel = 1** hay không.

Sau đó, dữ liệu được chia thành train, validation và test theo bệnh nhân, không chia theo từng dòng. Tỷ lệ chia là 70% cho train, 15% cho validation và 15% cho test. Quá trình chia có stratify theo trạng thái Sepsis của bệnh nhân, nhằm giữ tỷ lệ bệnh nhân Sepsis tương đối ổn định giữa các tập. Trong lần huấn luyện hiện tại, nhóm giữ toàn bộ bệnh nhân và không downsample nhóm không Sepsis.

Các thống kê dùng để chuẩn hóa chỉ được tính trên tập train. Với mỗi đặc trưng, nhóm tính trung bình và độ lệch chuẩn trên dữ liệu gốc bằng cách bỏ qua các giá trị

thiếu. Với các đặc trưng cần biến đổi log, nhóm tính thêm trung bình và độ lệch chuẩn sau khi log-transform. Việc chỉ tính thống kê trên tập train giúp tránh rò rỉ thông tin từ validation hoặc test vào quá trình huấn luyện.

Với dữ liệu thiếu, quy trình huấn luyện xử lý theo thứ tự sau. Trước hết, hệ thống tạo *missing mask*, trong đó giá trị 1 thể hiện chỉ số được đo thật và giá trị 0 thể hiện chỉ số bị thiếu. Sau đó, các giá trị thiếu được tạm thời thay bằng giá trị trung bình của từng biến trong tập train. Cuối cùng, hệ thống forward-fill theo thời gian trong từng bệnh nhân: nếu tại một giờ một chỉ số bị thiếu, giá trị gần nhất trước đó của chính chỉ số đó sẽ được dùng lại. Nếu ở đầu chuỗi chưa có giá trị trước đó, giá trị trung bình của tập train được giữ lại.

Một số biến xét nghiệm có phân phối lệch mạnh được log-transform trước khi chuẩn hóa, gồm AST, BUN, Alkalinephos, Creatinine, Bilirubin_direct, Lactate, Potassium, Bilirubin_total, TroponinI, PTT, WBC và Fibrinogen. Trước khi lấy log, giá trị được chặn dưới ở mức rất nhỏ để tránh lỗi khi gặp giá trị bằng 0. Việc log-transform giúp nén các giá trị quá lớn và giảm ảnh hưởng của ngoại lệ. Các biến còn lại được chuẩn hóa trực tiếp bằng trung bình và độ lệch chuẩn của tập train.

Hai công thức chuẩn hóa được sử dụng là:

$$x_{\text{norm}} = 10 \times \frac{x - \mu}{\sigma}$$

và với các biến log-transform:

$$x_{\text{norm}} = 10 \times \frac{\log(\max(x, 10^{-6})) - \mu_{\log}}{\sigma_{\log}}$$

Trong đó, μ , σ , $\mu_{\log}$ và $\sigma_{\log}$ đều được lấy từ thống kê của tập train.

5.4 Tạo đặc trưng cho mô hình

Sau khi xử lý thiếu dữ liệu, mỗi time-step được mở rộng thành nhiều nhóm đặc trưng hơn. Đây là điểm khác với việc chỉ đưa 40 chỉ số gốc vào mô hình. Quy trình này tạo đặc trưng theo từng bệnh nhân để giữ được thông tin theo thời gian, sau đó mới gộp dữ liệu thành ma trận huấn luyện cho XGBoost.

Với 40 biến gốc, nhóm tạo ra ba nhóm đặc trưng tại mỗi giờ:

- **Giá trị đã chuẩn hóa:** biểu diễn trạng thái hiện tại của bệnh nhân sau khi xử lý thiếu dữ liệu, log-transform với một số biến và chuẩn hóa theo thống kê của tập train.
- **Delta theo thời gian:** biểu diễn mức thay đổi của từng chỉ số so với giờ trước đó. Delta được tính trên dữ liệu gốc đã được điền thiếu, trước khi chuẩn hóa.
- **Missing mask:** cho biết giá trị tại vị trí đó là giá trị được ghi nhận thật hay được điền do thiếu dữ liệu. Nhờ đó, mô hình biết thêm mức độ đầy đủ của dữ liệu, chứ không chỉ nhìn vào giá trị sau khi đã điền.

Như vậy, mỗi giờ dữ liệu có $40 \times 3 = 120$ đặc trưng. Sau đó, hệ thống tạo cửa sổ thời gian gồm 6 giờ gần nhất. Mỗi mẫu đầu vào của XGBoost có:

$$120 \times 6 = 720$$

đặc trưng. Nếu một bệnh nhân chưa có đủ 6 giờ dữ liệu, phần còn thiếu được điền bằng 0 để đảm bảo kích thước đầu vào cố định.

Cửa sổ thời gian được xây dựng theo hướng lấy giờ hiện tại trước, sau đó lấy lần lượt các giờ trước đó. Ví dụ, tại thời điểm t , mẫu đầu vào gồm đặc trưng của t , $t - 1$, $t - 2$, cho đến tối đa 6 giờ. Cách tạo đặc trưng này giúp mô hình không chỉ nhìn vào trạng thái hiện tại, mà còn nhìn vào xu hướng thay đổi trong vài giờ gần nhất. Đây là điểm quan trọng với bài toán dự đoán sớm Sepsis, vì các dấu hiệu cảnh báo thường xuất hiện dần theo thời gian.

5.5 Huấn luyện và đánh giá mô hình

Mô hình được huấn luyện bằng XGBoost với mục tiêu `binary:logistic`, tức là dự đoán xác suất thuộc lớp Sepsis tại từng time-step. Các metric theo dõi trong quá trình train gồm logloss, AUC và AUCPR. Trong bối cảnh dữ liệu mất cân bằng, AUCPR có ý nghĩa hơn accuracy vì tập trung vào khả năng phát hiện lớp dương hiếm.

Do tỷ lệ nhân dương rất thấp, nhóm sử dụng tham số `scale_pos_weight`. Giá trị này được tính từ tỷ lệ giữa số mẫu âm và số mẫu dương trong tập train. Trong lần huấn luyện được export, `scale_pos_weight` có giá trị khoảng 54.54. Điều này giúp mô hình chú ý nhiều hơn đến các time-step có nhãn Sepsis.

Mô hình được huấn luyện với early stopping trên tập validation. Early stopping giúp dừng huấn luyện khi mô hình không còn cải thiện rõ rệt trên validation, từ đó hạn chế overfitting. Trong lần huấn luyện hiện tại, mô hình đạt best iteration tại vòng 381.

Sau khi có xác suất dự đoán, nhóm không dùng ngưỡng mặc định 0.5 một cách cứng nhắc. Thay vào đó, ngưỡng dự đoán được tìm bằng cách tối ưu normalized utility theo quy tắc của PhysioNet/CinC Challenge. Ngưỡng tốt nhất được chọn là:

$$\text{Optimal threshold} = 0.53$$

Bảng 2 trình bày kết quả đánh giá chính của mô hình trên validation và test.

Metric	Validation	Test
ROC-AUC	0.8303	0.8444
Average Precision	0.0943	0.1311
Recall	0.6367	0.6617
Precision	0.0693	0.0707
F1-score	0.1250	0.1277
Normalized Utility	0.3963	0.4187

Bảng 2: Kết quả đánh giá mô hình XGBoost

Các kết quả này phản ánh đúng tính chất khó của bài toán. Precision không cao do dữ liệu mất cân bằng mạnh và hệ thống ưu tiên phát hiện sớm nguy cơ Sepsis. Ngược lại, recall và normalized utility được quan tâm nhiều hơn vì mục tiêu chính là giảm khả năng bỏ sót các trường hợp nguy hiểm.

5.6 Mô hình export sẵn từ Kaggle

Sau khi huấn luyện trên Kaggle, nhóm export mô hình và các thông tin cần thiết để đưa vào hệ thống realtime. Bản export này không chỉ gồm mô hình XGBoost, mà còn có các file mô tả cách tiền xử lý dữ liệu, cách chia tập dữ liệu và thông tin môi trường để có thể nạp mô hình theo chuẩn MLflow.

Các file quan trọng trong bản export gồm:

- `model.json`: mô hình XGBoost đã huấn luyện.
- `scaler.json`: thống kê dùng cho xử lý thiếu dữ liệu, log-transform và chuẩn hóa. File này lưu bốn nhóm thống kê gồm mean, standard deviation, log-mean và log-standard deviation cho 40 đặc trưng.
- `metadata.json`: thông tin về thứ tự đặc trưng, các cột log-transform, kích thước cửa sổ, threshold, tham số huấn luyện và metric đánh giá.
- `splits.json`: danh sách bệnh nhân được dùng cho train, validation và test.
- `MLmodel`: file mô tả mô hình theo định dạng MLflow, trong đó khai báo XGBoost flavor và Python function flavor.
- `conda.yaml`, `python_env.yaml`, `requirements.txt`: thông tin môi trường chạy mô hình.

Trong `metadata.json`, mô hình được mô tả với 40 đặc trưng gốc, 120 đặc trưng sau khi tạo thêm delta và missing mask cho mỗi giờ, cửa sổ thời gian 6 giờ và tổng cộng 720 đặc trưng đầu vào. File này cũng lưu ngưỡng dự đoán tốt nhất là 0.53, best iteration là 381 và các metric đánh giá trên validation/test. Nhờ đó, khi triển khai, hệ thống không cần đoán lại cấu hình của mô hình mà có thể đọc trực tiếp từ `metadata`.

Khi hệ thống Spark Streaming chạy realtime, mô hình có thể được nạp trực tiếp từ bản export này. Điều này giúp hệ thống có thể hoạt động ngay cả khi chưa có model mới từ MLflow. Đồng thời, việc lưu `scaler.json` và `metadata.json` giúp đảm bảo pipeline realtime xử lý dữ liệu đúng như lúc huấn luyện. File `splits.json` cũng giúp nhóm kiểm tra lại danh sách bệnh nhân đã dùng cho từng tập, tránh nhầm lẫn khi so sánh kết quả giữa các lần train.

Việc export thêm `MLmodel` và các file môi trường cho thấy mô hình đã được đóng gói theo hướng có thể quản lý bằng MLflow. Nhờ đó, cùng một mô hình có thể được dùng theo hai cách: nạp trực tiếp từ file local trong pipeline realtime hoặc đăng ký/quản lý thông qua MLflow trong quy trình MLOps.

5.7 Quy trình huấn luyện lại với Airflow và MLflow

Bên cạnh mô hình export sẵn, hệ thống còn có quy trình huấn luyện lại khi có thêm dữ liệu mới. Airflow được dùng để lập lịch chạy quá trình huấn luyện, còn MLflow được dùng để lưu lại thông tin của từng lần train.

Khi quy trình huấn luyện lại được kích hoạt, hệ thống đọc dữ liệu huấn luyện từ các nguồn như dữ liệu train ban đầu, dữ liệu live dùng cho demo và dữ liệu lịch sử đã được Spark ghi vào HDFS. Sau đó, dữ liệu được xử lý theo cùng logic với mô hình ban đầu: xử lý thiếu dữ liệu, chuẩn hóa, tạo đặc trưng, tạo cửa sổ thời gian và huấn luyện XGBoost.

Trong quá trình huấn luyện, MLflow lưu lại các metric, tham số, scaler, mô hình và các artifact như biểu đồ utility hoặc ROC/PR curve. Việc lưu vết này giúp nhóm theo dõi chất lượng mô hình qua nhiều lần huấn luyện, so sánh các phiên bản và quyết định mô hình nào nên được sử dụng trong pipeline realtime.

Quy trình này giúp hệ thống không chỉ chạy với một mô hình cố định, mà có khả năng cập nhật khi dữ liệu mới xuất hiện. Đây là một phần quan trọng của MLOps trong dự án, giúp mô hình duy trì khả năng dự đoán tốt hơn theo thời gian.

6 Triển khai và vận hành hệ thống

6.1 Môi trường triển khai

Hệ thống được triển khai theo hướng container hóa bằng Docker Compose. Mỗi thành phần trong kiến trúc được chạy trong một container riêng, ví dụ như HDFS, Kafka, NiFi, Spark, Cassandra, Flask API, Nginx, Airflow và MLflow. Cách triển khai này giúp nhóm dễ khởi động toàn bộ hệ thống trên môi trường local, đồng thời hạn chế lỗi do khác biệt môi trường giữa các máy.

Máy chạy hệ thống cần cài đặt Docker và Docker Compose V2. Do hệ thống gồm nhiều dịch vụ Big Data chạy cùng lúc, tài nguyên khuyến nghị là tối thiểu 8GB RAM, tốt hơn là 16GB RAM và từ 4 CPU cores trở lên. Khi chạy lần đầu, Docker cần build các image nội bộ và tải thêm các image từ Docker Hub, vì vậy thời gian khởi động có thể lâu hơn so với các lần chạy sau.

Trước khi khởi chạy, nhóm cấu hình một số biến môi trường trong file `.env`. Các biến này điều khiển dữ liệu demo và tốc độ mô phỏng luồng dữ liệu:

- **ACTIVE_DATA_DIR**: thư mục dữ liệu mà NiFi sẽ quét để đưa vào pipeline. Trong dự án, dữ liệu demo được lấy từ `/data/live_data`.
- **DELAY**: khoảng nghỉ giữa các lần NiFi đẩy dữ liệu, giúp mô phỏng dữ liệu đi vào hệ thống theo thời gian.
- **USE_X_PATIENTS**: chế độ chọn bệnh nhân demo. Khi đặt là `false`, hệ thống chạy theo danh sách bệnh nhân mẫu.
- **MAX_PATIENTS**: giới hạn số bệnh nhân dùng trong chế độ demo hoặc kiểm thử tải.

6.2 Quy trình khởi chạy hệ thống

Để khởi chạy hệ thống, người dùng mở terminal tại thư mục gốc của dự án và chạy lệnh:

```
docker compose up -d -build
```

Lệnh này thực hiện build các image cần thiết và khởi động toàn bộ dịch vụ ở chế độ nền. Sau khi chạy lệnh, cần chờ một khoảng thời gian để các container lần lượt sẵn sàng. Một số container chỉ chạy để khởi tạo hệ thống, ví dụ như khởi tạo topic Kafka, tạo bảng Cassandra, tạo thư mục HDFS hoặc cấu hình NiFi pipeline. Các container này hoàn thành nhiệm vụ rồi thoát với trạng thái thành công.

Sau khi hệ thống khởi động, có thể kiểm tra trạng thái bằng lệnh:

```
docker compose ps
```

Khi hệ thống hoạt động đúng, các dịch vụ chính như Kafka, NiFi, Spark, Cassandra, API, Nginx, Airflow và MLflow sẽ ở trạng thái **Up** hoặc **Healthy**. Các dịch vụ khởi tạo một lần như `kafka-init`, `cassandra-init`, `hdfs-init`, `nifi-setup` và `airflow-init` có thể ở trạng thái **Exited** (0), nghĩa là đã chạy xong nhiệm vụ cấu hình ban đầu.

Trong quá trình chạy, dữ liệu đi qua các bước chính như sau. NiFi đọc dữ liệu PSV trong thư mục live data, chuyển sang JSON và gửi vào Kafka topic `icu_data`. Spark Streaming đọc dữ liệu từ Kafka, xử lý theo từng bệnh nhân, nạp mô hình XGBoost để dự đoán nguy cơ Sepsis và ghi kết quả vào Cassandra/HDFS. Flask API đọc dữ liệu đã xử lý và trả về cho dashboard thông qua Nginx.

6.3 Theo dõi các dashboard trong hệ thống

Sau khi các container đã chạy ổn định, người dùng có thể truy cập các dashboard và giao diện quản trị của hệ thống. Bảng 3 liệt kê các dashboard chính.

Dashboard	Đường dẫn	Chức năng
Sepsis Dashboard	http://localhost:5000/dashboard	Theo dõi chỉ số ICU, xác suất Sepsis và trạng thái cảnh báo.
Apache NiFi	https://localhost:8443/nifi	Theo dõi pipeline đọc dữ liệu, chuyển đổi PSV sang JSON và gửi vào Kafka.
MLflow Tracking UI	http://localhost:5001	Theo dõi các lần huấn luyện mô hình, metric và artifact.
Apache Airflow UI	http://localhost:8081	Theo dõi và kích hoạt DAG huấn luyện lại mô hình.
Spark Master UI	http://localhost:4040	Theo dõi trạng thái xử lý streaming của Spark.

Bảng 3: Các dashboard và giao diện truy cập khi hệ thống đang chạy

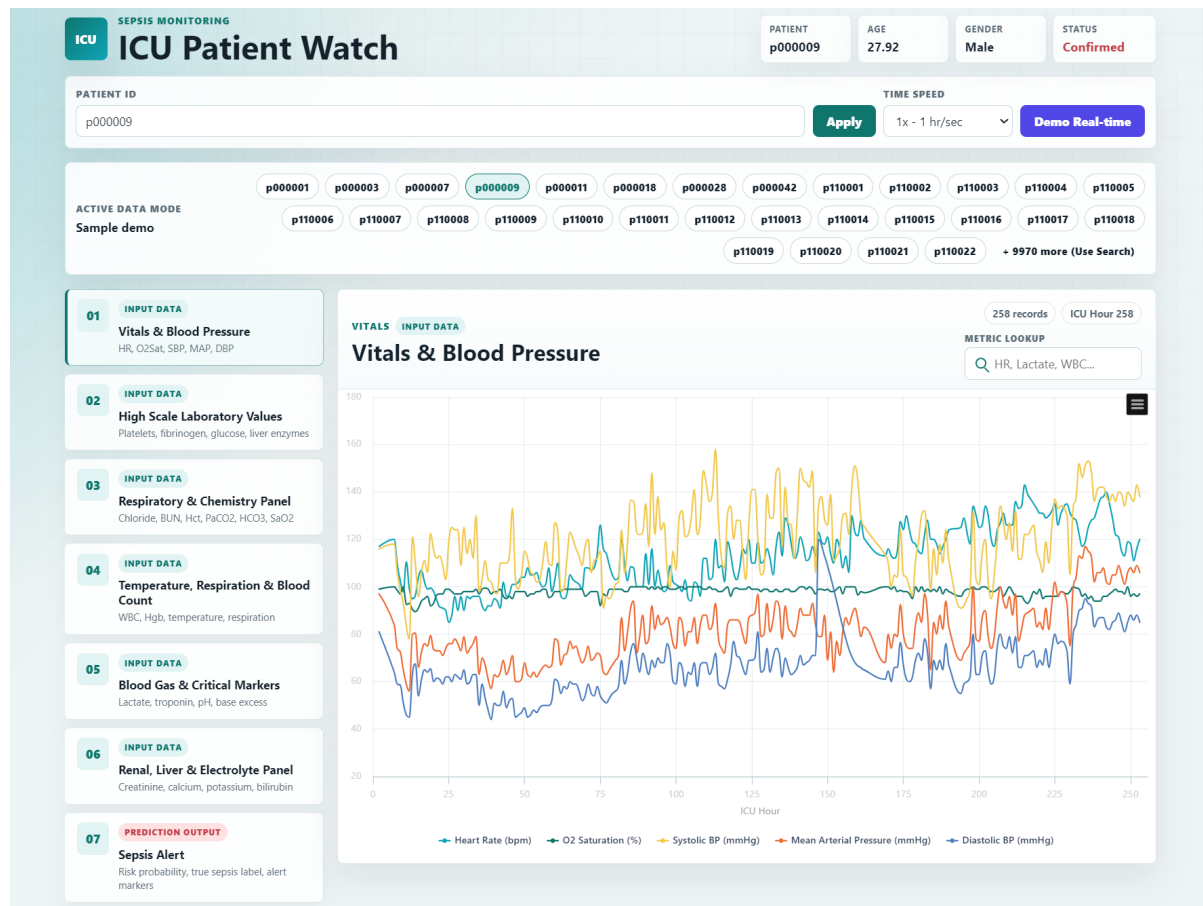
Trong các giao diện trên, dashboard là phần dành cho người dùng cuối. NiFi, Spark, Airflow và MLflow chủ yếu phục vụ nhóm triển khai để kiểm tra pipeline, theo dõi trạng thái xử lý và quản lý mô hình. Trong cấu hình hiện tại, Nginx mở cổng 5000 và chuyển request đến các Flask API replica phía sau. Nhờ đó, người dùng chỉ cần truy cập một địa chỉ dashboard duy nhất.

6.4 Vận hành dashboard và demo hệ thống

Dashboard của hệ thống có tên *ICU Patient Watch*. Giao diện cho phép người dùng nhập mã bệnh nhân, chọn bệnh nhân mẫu, tải dữ liệu hiện có và chạy chế độ mô phỏng realtime. Khi nhấn *Demo Real-time*, dashboard lần lượt hiển thị thêm dữ liệu theo từng giờ ICU, mô phỏng quá trình bệnh nhân được theo dõi liên tục.

Trên dashboard, phần trên cùng hiển thị thông tin tổng quan của bệnh nhân như mã bệnh nhân, tuổi, giới tính và trạng thái cảnh báo hiện tại. Bên dưới là vùng tìm kiếm

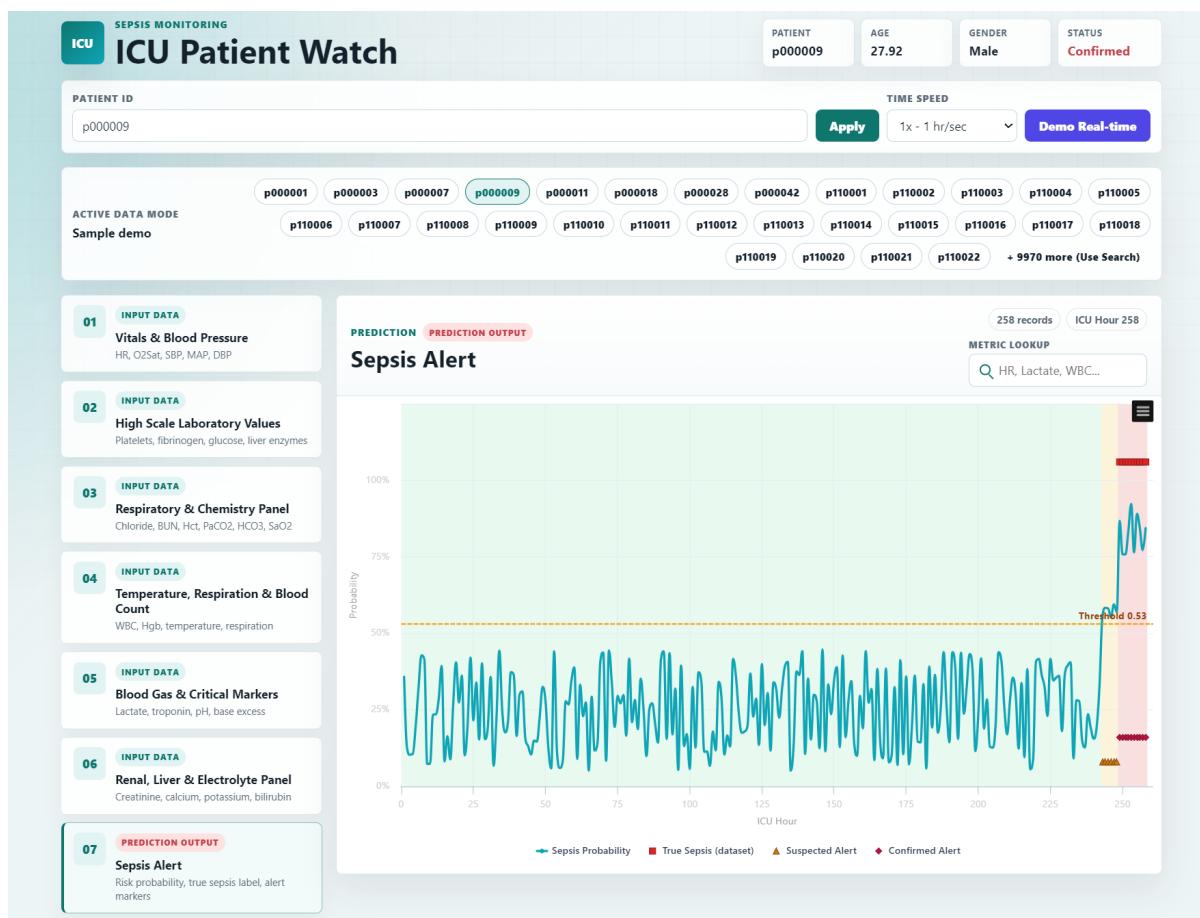
bệnh nhân, nút *Apply*, lựa chọn tốc độ thời gian và nút chạy demo realtime. Khu vực bên trái là danh sách các nhóm chỉ số, được chia thành nhiều tab như dấu hiệu sinh tồn, xét nghiệm thang đo lớn, hô hấp và hóa sinh, nhiệt độ và công thức máu, khí máu và marker nguy hiểm, chức năng thận/gan/điện giải, cùng tab dự đoán Sepsis.



Hình 13: Giao diện dashboard theo dõi dấu hiệu sinh tồn và huyết áp

Hình 13 minh họa tab theo dõi dấu hiệu sinh tồn và huyết áp. Biểu đồ hiển thị các chỉ số như nhịp tim, nồng độ oxy máu, huyết áp tâm thu, huyết áp trung bình và huyết áp tâm trương theo trục thời gian ICU. Cách hiển thị theo chuỗi thời gian giúp người dùng quan sát xu hướng thay đổi của bệnh nhân, thay vì chỉ nhìn một giá trị đơn lẻ.

Ngoài các chỉ số đầu vào, dashboard còn có tab riêng cho kết quả dự đoán Sepsis. Đây là phần quan trọng nhất đối với mục tiêu cảnh báo sớm của hệ thống.



Hình 14: Giao diện dashboard hiển thị xác suất dự đoán Sepsis

Hình 14 hiển thị xác suất Sepsis theo thời gian. Đường màu xanh biểu diễn xác suất dự đoán của mô hình. Đường ngưỡng được đặt tại 0.53, tương ứng với ngưỡng tối ưu đã chọn trong quá trình huấn luyện. Khi xác suất vượt ngưỡng và tín hiệu cảnh báo kéo dài, hệ thống hiển thị các marker cảnh báo như *Suspected Alert* và *Confirmed Alert*. Nếu dữ liệu có nhãn thật Sepsis, dashboard cũng hiển thị marker *True Sepsis* để người dùng so sánh thời điểm mô hình cảnh báo với nhãn trong dataset.

Nhờ cách trực quan hóa này, người dùng có thể theo dõi đồng thời hai nhóm thông tin: dữ liệu lâm sàng gốc của bệnh nhân và kết quả phân tích của mô hình AI. Điều này giúp dashboard không chỉ là nơi xem biểu đồ, mà còn là giao diện thể hiện kết quả của toàn bộ pipeline Big Data từ dữ liệu đầu vào, xử lý streaming, lưu trữ đến dự đoán Sepsis.

6.5 Dừng và reset hệ thống

Khi muốn dừng hệ thống tạm thời nhưng vẫn giữ dữ liệu trong volume, có thể sử dụng lệnh:

```
docker compose stop
```

Lệnh này dừng các container nhưng không xóa dữ liệu của Cassandra, HDFS, Postgres, MLflow và checkpoint của Spark. Khi cần chạy lại, người dùng có thể dùng lại lệnh `docker compose up -d`.

Trong trường hợp muốn reset toàn bộ hệ thống và xóa sạch dữ liệu sinh ra trong quá trình chạy, có thể dùng lệnh:

```
docker compose down -v
```

Cần lưu ý rằng tùy chọn `-v` sẽ xóa các Docker volume, bao gồm dữ liệu Cassandra, dữ liệu HDFS, metadata Airflow/Postgres, artifact MLflow và checkpoint của Spark. Sau khi reset, hệ thống sẽ cần khởi tạo lại các thành phần và xử lý lại dữ liệu từ đầu.

7 Kết quả và đánh giá

7.1 Kết quả xây dựng hệ thống

Sau quá trình triển khai, nhóm đã xây dựng được một hệ thống Big Data hoàn chỉnh cho bài toán giám sát bệnh nhân ICU và cảnh báo nguy cơ Sepsis. Hệ thống không chỉ dừng ở bước huấn luyện mô hình, mà còn có đầy đủ các thành phần từ thu thập dữ liệu, truyền dữ liệu, xử lý streaming, lưu trữ, cung cấp API đến trực quan hóa kết quả trên dashboard.

Về luồng dữ liệu realtime, hệ thống có thể đọc dữ liệu bệnh nhân dạng PSV từ thư mục demo, chuyển đổi dữ liệu sang JSON và gửi vào Kafka. Spark Streaming tiếp tục đọc dữ liệu từ Kafka, xử lý theo từng bệnh nhân, tạo đặc trưng theo chuỗi thời gian và sử dụng mô hình XGBoost để dự đoán xác suất Sepsis. Kết quả xử lý được ghi vào Cassandra để phục vụ truy vấn nhanh, đồng thời lưu vào HDFS dưới dạng Parquet để lưu trữ lịch sử.

Về tầng phục vụ dữ liệu, Flask API đã được xây dựng để dashboard có thể truy vấn dữ liệu theo mã bệnh nhân. API ưu tiên đọc từ Cassandra, sau đó fallback sang HDFS hoặc file PSV cục bộ khi cần. Nginx được dùng làm lớp trung gian phía trước API, giúp người dùng truy cập dashboard qua một địa chỉ thống nhất.

Về giao diện người dùng, dashboard *ICU Patient Watch* đã hiển thị được các nhóm chỉ số ICU theo thời gian như dấu hiệu sinh tồn, huyết áp, xét nghiệm, chỉ số hô hấp, chỉ số hóa sinh và xác suất Sepsis. Người dùng có thể chọn bệnh nhân, xem dữ liệu theo từng giờ ICU và chạy chế độ mô phỏng realtime để quan sát diễn biến dữ liệu.

7.2 Kết quả mô hình dự đoán Sepsis

Mô hình XGBoost được huấn luyện trên bộ dữ liệu PhysioNet/CinC Challenge 2019 với đầu vào là các cửa sổ dữ liệu 6 giờ. Mỗi giờ dữ liệu gồm 40 biến gốc, sau đó được mở rộng thành 120 đặc trưng nhờ kết hợp giá trị đã chuẩn hóa, delta theo thời gian và missing mask. Vì vậy, mỗi mẫu đầu vào của mô hình có 720 đặc trưng.

Kết quả đánh giá mô hình đã được trình bày ở Bảng 2. Trên tập test, mô hình đạt ROC-AUC khoảng 0.8444, Average Precision khoảng 0.1311, Recall khoảng 0.6617 và Normalized Utility khoảng 0.4187. Các kết quả này cho thấy mô hình đã học được một phần tín hiệu liên quan đến nguy cơ Sepsis trong dữ liệu ICU.

Do dữ liệu có mức mất cân bằng lớn, Precision của mô hình còn thấp. Điều này có nghĩa là hệ thống vẫn có thể sinh ra một số cảnh báo giả. Tuy nhiên, trong bài toán cảnh báo sớm Sepsis, nhóm ưu tiên khả năng phát hiện các ca nguy cơ hơn là chỉ tối ưu

độ chính xác tổng thể. Vì vậy, Recall và Normalized Utility được xem là hai chỉ số quan trọng hơn trong quá trình đánh giá.

Ngưỡng dự đoán được chọn là 0.53 thay vì dùng mặc định 0.5. Ngưỡng này được tối ưu theo normalized utility, phù hợp với tinh thần của bài toán PhysioNet/CinC Challenge: cảnh báo đúng và đủ sớm sẽ có giá trị cao hơn cảnh báo trễ, còn cảnh báo sai cũng cần được kiểm soát để tránh gây nhiễu cho người dùng.

7.3 Kết quả demo trên dashboard

Trong quá trình demo, dashboard đã hiển thị được dữ liệu theo từng bệnh nhân và theo từng giờ ICU. Hình 13 cho thấy giao diện theo dõi các chỉ số sinh tồn và huyết áp. Người dùng có thể quan sát nhịp tim, nồng độ oxy máu, huyết áp tâm thu, huyết áp trung bình và huyết áp tâm trương trên cùng một biểu đồ. Cách hiển thị này giúp việc theo dõi xu hướng trở nên trực quan hơn so với việc chỉ xem dữ liệu dạng bảng.

Hình 14 cho thấy phần dự đoán Sepsis trên dashboard. Biểu đồ hiển thị xác suất Sepsis theo thời gian, đường ngưỡng 0.53, nhân thật Sepsis trong dataset và các marker cảnh báo. Khi xác suất dự đoán tăng cao và vượt ngưỡng, dashboard chuyển trạng thái cảnh báo sang mức nghi ngờ hoặc xác nhận. Đây là phần thể hiện rõ nhất kết quả kết nối giữa mô hình AI và hệ thống trực quan hóa.

Ngoài hai hình minh họa chính, dashboard còn có các tab khác cho từng nhóm chỉ số chuyên môn. Việc chia tab giúp giao diện không bị quá tải thông tin, đồng thời người dùng vẫn có thể chuyển nhanh giữa các nhóm dữ liệu khi cần kiểm tra sâu hơn.

7.4 Đánh giá ưu điểm và hạn chế

Về ưu điểm, hệ thống đã thể hiện được một pipeline Big Data tương đối đầy đủ. Các thành phần được tách nhiệm vụ rõ ràng: NiFi phụ trách thu thập và chuyển đổi dữ liệu, Kafka đóng vai trò message broker, Spark xử lý streaming và dự đoán, Cassandra/HDFS lưu trữ dữ liệu, Flask API phục vụ truy vấn, dashboard hiển thị kết quả, còn Airflow/MLflow hỗ trợ quy trình MLOps. Cách thiết kế này giúp hệ thống dễ theo dõi và dễ mở rộng hơn so với việc gộp toàn bộ logic vào một chương trình duy nhất.

Một điểm tích cực khác là quy trình xử lý dữ liệu realtime được giữ thống nhất với quy trình huấn luyện mô hình. Thứ tự đặc trưng, cách xử lý thiếu dữ liệu, log-transform, missing mask, delta và cửa sổ 6 giờ đều được đồng bộ giữa lúc train và lúc inference. Điều này giúp giảm rủi ro mô hình nhận dữ liệu sai định dạng khi đưa vào pipeline streaming.

Hệ thống cũng có cơ chế fallback ở tầng API. Khi Cassandra chưa có dữ liệu hoặc dữ liệu không còn do TTL, API có thể đọc từ HDFS hoặc file PSV cục bộ để dashboard vẫn có dữ liệu hiển thị trong quá trình demo. Đây là điểm hữu ích khi trình bày hệ thống hoặc kiểm thử từng thành phần.

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế. Thứ nhất, mô hình vẫn còn bị ảnh hưởng bởi dữ liệu mất cân bằng, thể hiện qua Precision thấp. Thứ hai, hệ thống mới được kiểm thử trong môi trường local bằng Docker Compose, chưa được triển khai trên cụm phân tán thật với nhiều node vật lý. Thứ ba, dashboard hiện phục vụ mục tiêu demo và giám sát trực quan, chưa có các chức năng nâng cao như phân quyền người dùng, ghi nhận phản hồi của bác sĩ hoặc xác nhận cảnh báo thủ công.

Ngoài ra, dữ liệu sử dụng trong dự án là dữ liệu nghiên cứu công khai, không phải dữ liệu được lấy trực tiếp từ bệnh viện theo thời gian thực. Vì vậy, kết quả của hệ thống

nên được hiểu là mô phỏng kỹ thuật và hỗ trợ nghiên cứu, chưa thể dùng như một công cụ y tế thực tế nếu chưa có kiểm định lâm sàng đầy đủ.

8 Kết luận và hướng phát triển

8.1 Kết luận

Trong dự án này, nhóm đã xây dựng một hệ thống Big Data phục vụ giám sát bệnh nhân ICU và cảnh báo sớm nguy cơ Sepsis. Hệ thống mô phỏng được luồng dữ liệu bệnh nhân từ file PSV, đưa dữ liệu vào pipeline streaming, xử lý theo từng bệnh nhân, dự đoán xác suất Sepsis bằng mô hình XGBoost và hiển thị kết quả trên dashboard.

Về mặt kiến trúc, dự án đã kết hợp nhiều công nghệ trong hệ sinh thái Big Data như Apache NiFi, Apache Kafka, Apache Spark Streaming, Apache Cassandra, HDFS, Flask API, Nginx, Airflow và MLflow. Mỗi công nghệ đảm nhiệm một phần riêng trong hệ thống, tạo thành một quy trình tương đối khép kín từ dữ liệu đầu vào đến kết quả hiển thị cho người dùng.

Về mặt mô hình, nhóm đã huấn luyện và export được mô hình XGBoost từ môi trường Kaggle. Mô hình sử dụng dữ liệu chuỗi thời gian theo cửa sổ 6 giờ, kết hợp giá trị đã chuẩn hóa, delta theo thời gian và missing mask. Cách tạo đặc trưng này giúp mô hình khai thác được cả trạng thái hiện tại và xu hướng thay đổi gần đây của bệnh nhân.

Về mặt triển khai, hệ thống đã được đóng gói bằng Docker Compose, giúp việc khởi chạy các thành phần trở nên thuận tiện hơn. Dashboard đã hiển thị được các chỉ số ICU, xác suất Sepsis, ngưỡng dự đoán và trạng thái cảnh báo. Điều này cho thấy hệ thống không chỉ có mô hình dự đoán riêng lẻ, mà đã có một pipeline hoàn chỉnh để đưa kết quả mô hình đến giao diện theo dõi.

Nhìn chung, dự án đã đạt được mục tiêu đề ra: xây dựng một hệ thống Big Data mô phỏng giám sát bệnh nhân ICU gần thời gian thực và hỗ trợ cảnh báo sớm nguy cơ Sepsis. Tuy nhiên, hệ thống hiện vẫn mang tính mô phỏng và nghiên cứu, chưa thay thế được quyết định chuyên môn của bác sĩ trong môi trường lâm sàng thật.

8.2 Hướng phát triển

Trong tương lai, hệ thống có thể được phát triển thêm theo một số hướng sau:

- **Cải thiện mô hình dự đoán:** thử nghiệm thêm các mô hình khác như LightGBM, CatBoost, LSTM, GRU hoặc Transformer cho dữ liệu chuỗi thời gian. Các mô hình này có thể giúp khai thác tốt hơn quan hệ dài hạn giữa các chỉ số ICU.
- **Mở rộng đặc trưng thời gian:** bổ sung các đặc trưng như trung bình trượt, độ dốc thay đổi, độ biến động trong nhiều cửa sổ thời gian khác nhau hoặc thời gian kể từ lần đo gần nhất của từng chỉ số.
- **Giảm cảnh báo giả:** tinh chỉnh ngưỡng cảnh báo, thử các chiến lược xử lý mất cân bằng nâng cao và đánh giá thêm false alarm rate để dashboard đưa ra cảnh báo ổn định hơn.

- **Tăng khả năng giải thích mô hình:** tích hợp các phương pháp như SHAP để giải thích chỉ số nào ảnh hưởng nhiều đến dự đoán Sepsis tại từng thời điểm. Điều này giúp người dùng hiểu rõ hơn lý do hệ thống sinh cảnh báo.
- **Hoàn thiện dashboard:** bổ sung phân quyền người dùng, lịch sử cảnh báo, chức năng ghi chú của bác sĩ và cơ chế xác nhận hoặc bỏ qua cảnh báo.
- **Cải thiện MLOps:** hoàn thiện quy trình huấn luyện lại mô hình bằng Airflow và MLflow, bổ sung kiểm tra drift dữ liệu, so sánh phiên bản mô hình và cơ chế rollback khi mô hình mới hoạt động kém hơn.

Phân công nhiệm vụ

Mã sinh viên	Họ và tên	Nhiệm vụ chính
23020358	Lê Thiện Đức	• Thu thập dữ liệu. • Xử lý và phân tích dữ liệu. • Xây dựng phần huấn luyện lại mô hình sử dụng dữ liệu từ HDFS. • Tham gia triển khai pipeline MLOps. • Đóng gói hệ thống bằng Docker. • Viết và chỉnh sửa báo cáo. • Làm slide thuyết trình.
23020376	Nguyễn Đức Huy	• Thu thập dữ liệu. • Xử lý và phân tích dữ liệu. • Làm giao diện dashboard. • Huấn luyện mô hình XGBoost trên Kaggle. • Export mô hình, scaler và metadata để tích hợp vào Spark Streaming. • Đóng gói hệ thống bằng Docker. • Viết và chỉnh sửa báo cáo. • Làm slide thuyết trình.

Bảng 4: Phân công nhiệm vụ của các thành viên

Tài liệu tham khảo

- [1] John Anda Du, Nadi Sadr, and Philip de Chazal. Automated Prediction of Sepsis Onset Using Gradient Boosted Decision Trees. In *2019 Computing in Cardiology Conference (CinC)*, volume 46, pages 1–4, 2019.
- [2] Matthew Reyna, Chris Josef, Russell Jeter, Supreeth Shashikumar, Benjamin Moody, M. Brandon Westover, Ashish Sharma, Shamim Nemati, and Gari D. Clifford. Early Prediction of Sepsis from Clinical Data: The PhysioNet/Computing in Cardiology Challenge 2019. *PhysioNet*, August 2019. Version 1.0.0.
- [3] Quan Zou, Sifa Xie, Ziyu Lin, Meihong Wu, and Ying Ju. Finding the Best Classification Threshold in Imbalanced Classification. *Big Data Research*, 5:2–8, 2016. Big data analytics and applications.